

RESTful API Assignment Submission Report

Assignment Name:	Build a RESTful API using Node.js and Express (100 Marks)
Project Folder:	express-rest-assignment
Tech Stack:	Node.js, Express.js, Thunder Client / Postman
Data Source:	In-Memory Array (as per assignment requirements)

1. Overview

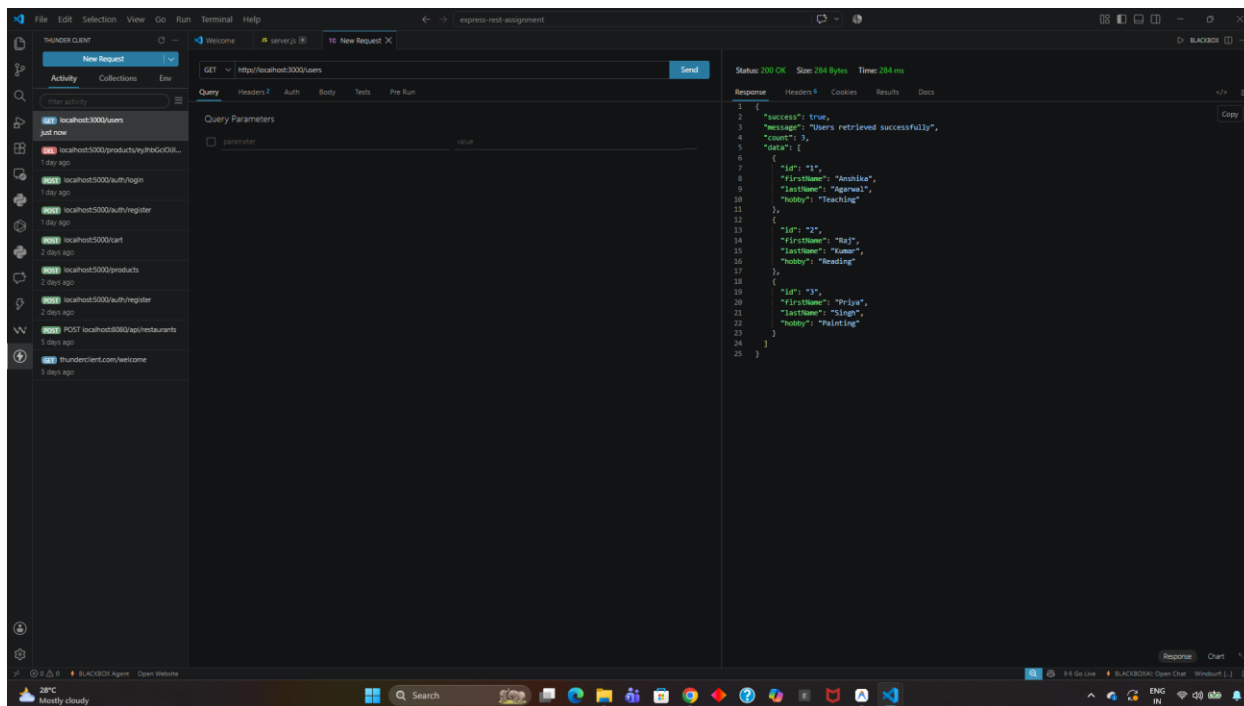
This report documents the implementation and testing of a RESTful API built using Node.js and Express. The API provides full CRUD operations for managing user records in an in-memory array. It incorporates logging middleware, request validation middleware, error handling, and standard HTTP status codes.

2. API Testing Screenshots & Verification

Test 1: GET /users (Fetch All Users)

URL: http://localhost:3000/users

Method: GET | **Status Code:** 200 OK



Test 2: GET /users/:id (Fetch User by ID)

URL: http://localhost:3000/users/1

Method: GET | **Status Code:** 200 OK

The screenshot displays the Thunder Client interface with a REST client configuration. The request is a GET method to the URL `http://localhost:3000/users/1`. The response status is 200 OK, with a size of 136 bytes and a time of 16 ms. The response body is a JSON object representing a user.

Request Details:

- Method: GET
- URL: `http://localhost:3000/users/1`
- Headers: 2
- Auth: None
- Body: Empty

Response Details:

- Status: 200 OK
- Size: 136 Bytes
- Time: 16 ms
- Response Body (JSON):

```
1 {
2   "success": true,
3   "message": "User retrieved successfully",
4   "data": {
5     "id": "1",
6     "firstName": "Anshika",
7     "lastName": "Agarwal",
8     "hobby": "Teaching"
9   }
10 }
```

The left sidebar shows a list of recent requests, including `localhost:3000/users` and `localhost:5000/products`. The bottom status bar indicates the system temperature is 28°C and the weather is mostly cloudy.

Test 3: POST /user (Add New User - Success)

URL: http://localhost:3000/user

Method: POST | **Status Code:** 201 Created

THUNDER CLIENT

File Edit Selection View Go Run Terminal Help

express-rest-assignment

server.js

New Request X

Activity Collections Env

Filter activity

POST localhost:3000/users just now

localhost:5000/products by /fbGcOIL... 1 day ago

localhost:5000/auth/login 1 day ago

localhost:5000/auth/register 1 day ago

localhost:5000/cart 2 days ago

localhost:5000/products 2 days ago

localhost:5000/auth/register 2 days ago

POST localhost:8080/api/restaurants 5 days ago

thunderClient.com/welcome 5 days ago

POST http://localhost:3000/user Send

Query Headers Auth Body Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content Format

```
1 {
2   "firstName": "Rahul",
3   "lastName": "Sharma",
4   "hobby": "Coding"
5 }
```

Status: 201 Created Size: 129 Bytes Time: 16 ms

Response Headers Cookies Results Docs

```
1 {
2   "success": true,
3   "message": "User created successfully",
4   "data": {
5     "id": "4",
6     "firstName": "Rahul",
7     "lastName": "Sharma",
8     "hobby": "Coding"
9   }
10 }
```

28°C Mostly cloudy

Search

Go Live BLACKBOX: Open Chat Windturf

Test 4: POST /user (Validation Error Case)

URL: http://localhost:3000/user

Method: POST | **Status Code:** 400 Bad Request

The screenshot shows the Thunder Client interface with a new request configured for POST to http://localhost:3000/user. The request body is a JSON object:

```
{  "firstName": "Rahul"}
```

. The response status is 400 Bad Request, with a size of 196 Bytes and a time of 13 ms. The response body is a JSON object:

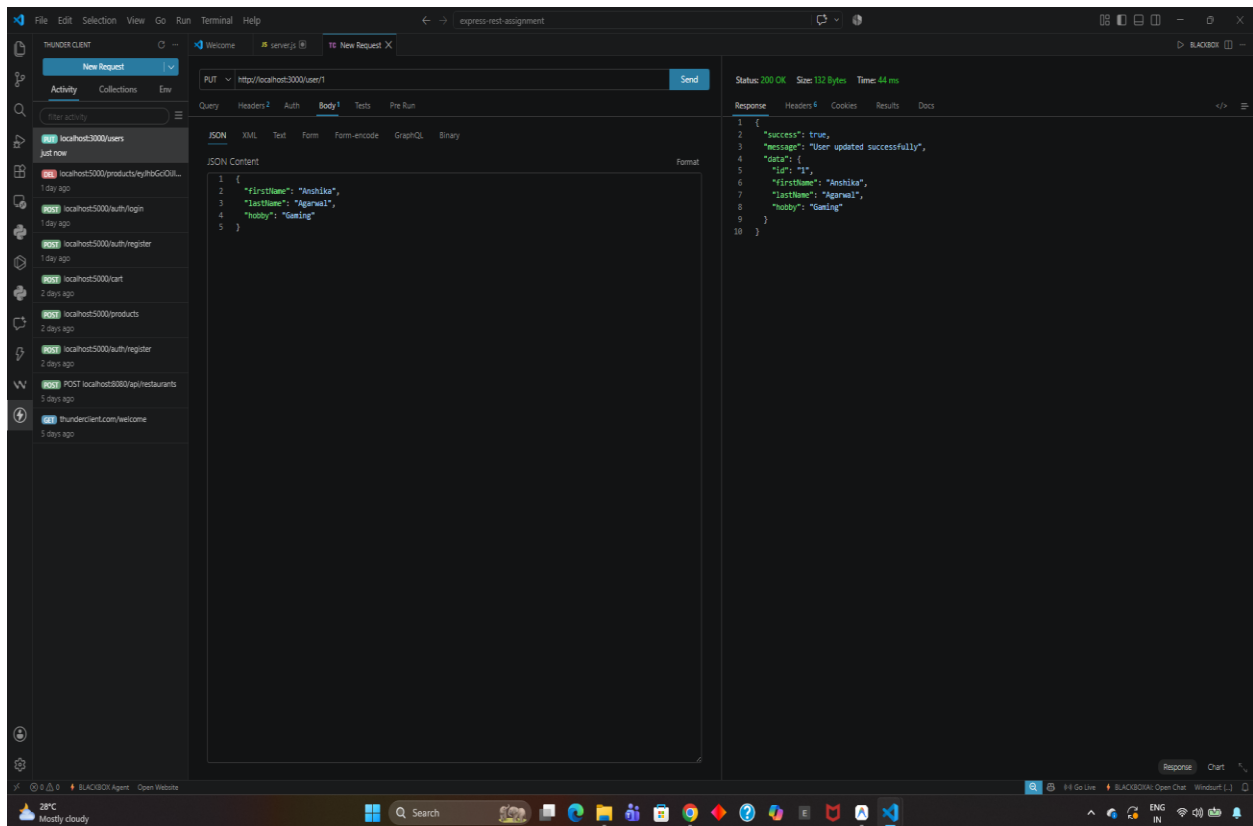
```
{  "error": "Validation Error",  "message": "All fields (firstName, lastName, hobby) are required and must not be empty",  "receivedFields": {    "firstName": "provided",    "lastName": "missing",    "hobby": "missing"  }}
```

. The left sidebar shows a list of requests, and the bottom status bar indicates the system temperature is 28°C and mostly cloudy.

Test 5: PUT /user/:id (Update User Details)

URL: http://localhost:3000/user/1

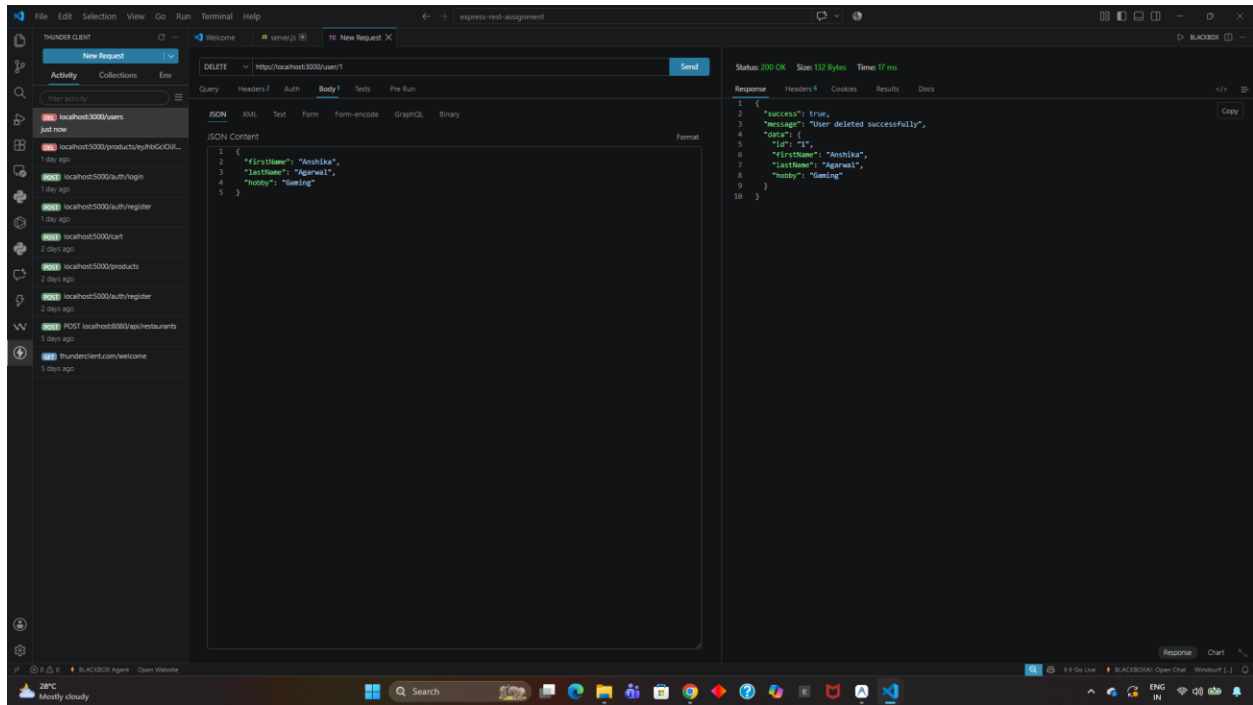
Method: PUT | **Status Code:** 200 OK



Test 6: DELETE /user/:id (Delete User)

URL: `http://localhost:3000/user/1`

Method: DELETE | **Status Code:** 200 OK



3. Conclusion

All API endpoints have been tested thoroughly using Thunder Client. The API handles CRUD operations properly, enforces data validation, logs requests, and responds with appropriate HTTP status codes.